Building a World Action Model from Scratch: Design Decisions, Lessons, and a Map of Open WAMs

Keivalya Pandya
Northeastern University

Abstract

A vision-language-action (VLA) policy learns what to do. A world action model (WAM) also learns what will happen when it does it. Today's WAMs are large video models that few people can train, which makes their design hard to study. We introduce **pico-WAM**, a 0.75M-parameter WAM written from scratch in about 1,000 lines of PyTorch, and use it to walk through the decisions every WAM makes: what to predict, how actions meet the future, which backbone to use, and what the prediction is for at test time. We place eight open-source WAMs on the same map. On a toy navigation world with a true simulator, pico-WAM's latent dynamics imagines 15 steps ahead with $\sim$80$\times$ less state error than persistence. Three lessons stand out. Expert-only data hides action effects: counterfactual error grows 15$\times$ while factual error even improves. Prediction helps the policy only when the observation hides state: with pixel input, success rises from 36% to 80%, and a separate matched pilot suggests that most of such a gain comes from state supervision rather than from dynamics. Pixels dominate the cost: the image decoder holds half the parameters and 90% of the training compute. Code, data and every figure's source are public.

1 From acting to predicting

A VLA maps an image, an instruction and the robot state to an action. Nothing in its training asks what the action will do. A world model answers exactly that question, but does not choose actions. A WAM does both with one network (Figure 1). Recent surveys define WAMs as models that "connect future-state modeling with executable action generation" [9] and sort them by the future they produce: pixels, latents, or none at all [10].

Open WAMs now reach billions of parameters [6–8], and Genie Envisioner pretrains on a million real robot episodes [8]. That scale buys capability, but it hides the basic questions behind compute. What does the dynamics part actually learn? When does predicting the future make the policy better? Which choices are expensive? Conceptual tutorials map this design space [12], and minimal codebases exist for video prediction [4], but neither lets you train a complete WAM and look inside it in an afternoon.

We build that WAM. Our contributions are:

- **A map of WAM design decisions**, with eight open-source WAMs placed on it (Figure 2, Table 1).
- **pico-WAM**, a from-scratch WAM in which each module added to the VLA is under 100 lines, plus a toy world with a true simulator for counterfactual tests (Section 3).

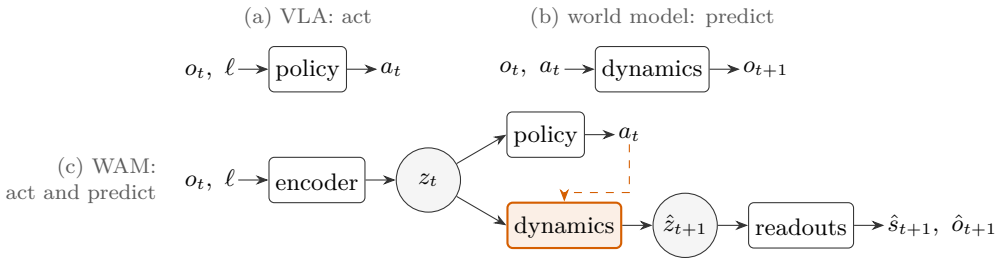


Figure 1: Three ways to use a model of the world. (a) A VLA maps the current observation o_t and instruction ℓ to an action. (b) A world model maps an observation and an action to the next observation; it predicts but never decides. (c) A WAM shares one representation z_t between a policy and an action-conditioned dynamics model (orange). The dynamics can be rolled forward with any action, so the model can be asked what its own action, or any other, will do. pico-WAM implements (c) with a 128-dimensional latent and reads predicted latents out as physical states and images.

- **A record of what worked and what did not**, from data and losses to decoder design, measured over independent runs (Section 4, Table 2).
- **A cost breakdown** that shows where a small WAM spends parameters and compute (Section 5).

2 The WAM design space

Every WAM answers four questions (Figure 2). Table 1 shows how eight widely used open-source systems answer them.

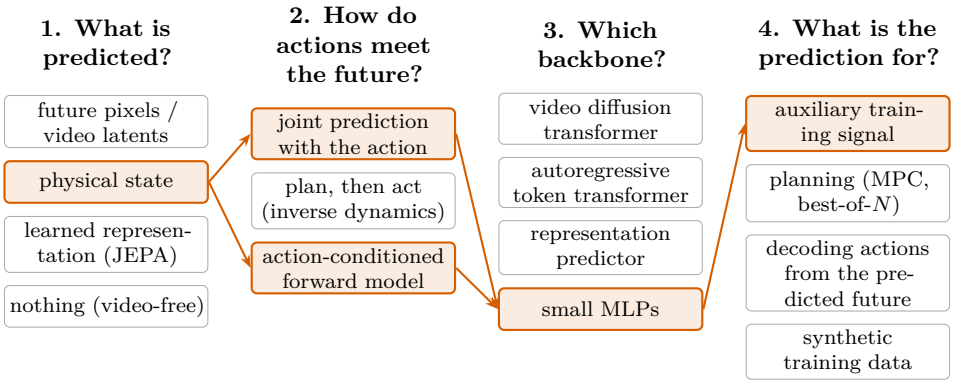


Figure 2: The four decisions behind every WAM, and pico-WAM's path through them (orange). (1) The prediction target ranges from pixels to compact learned representations, or no future at all. pico-WAM predicts the physical state (and, as an option, pixels). (2) Actions and futures can be generated together, the future can be generated first and an inverse-dynamics model can read the action off it, or an explicit forward model can take any action as input. pico-WAM combines joint training with an action-conditioned forward model. (3) Backbones range from billion-parameter video diffusion transformers to small MLPs. (4) The prediction can shape the representation during training, rank candidate actions at test time, produce the action directly, or generate data for another policy. pico-WAM uses it only as a training signal, the cheapest and most common choice. Table 1 places eight open-source WAMs on the same map.

System	Predicts	Backbone	Action decoding	Future used for
GR-1 [11]	future frames (image patches)	GPT-style causal transformer, video-pretrained on Ego4D clips	linear head on an `[ACT]` token	training signal
UWM [13]	video and actions	one diffusion transformer, independent timesteps per modality	action diffusion	policy, forward or inverse dynamics, video generation
WorldVLA [2]	next frame (VQ tokens)	autoregressive multimodal LLM (Chameleon init.)	discrete action tokens	training signal
Genie Envisioner [8]	video chunks	video diffusion transformer (2B)	flow-matching action module on video latents	action decoding; neural simulator
Cosmos Policy [6]	future states and values as latent frames	Cosmos-Predict2 video diffusion (2B)	actions injected as latent frames	best-of-N planning
V-JEPA 2-AC [1]	future representations	frozen V-JEPA 2 encoder + 300M action-conditioned predictor	none: actions come from the planner	CEM model-predictive control
DreamGen [5]	robot videos	fine-tuned video generation model	inverse dynamics or latent actions label the videos	synthetic training data
LingBot-VA [7]	video latents, autoregressively	dual-stream diffusion transformer (5.3B), KV cache	inverse dynamics from the predicted future	action decoding; memory
pico-WAM (ours)	physical state, pixels, latents	3-layer CNN + GRU + MLPs (0.75M)	MLP, diffusion or flow head	training signal; analysis

Table 1: Eight open-source WAMs on the map of Figure 2. All eight predict some form of the future; they differ in what they predict, how actions are produced, and what the prediction is used for at test time. Two trends are visible. Large systems inherit video-generation backbones and predict latent frames rather than raw pixels. Systems that plan (Cosmos Policy, V-JEPA 2-AC) or decode actions from the future (Genie Envisioner, LingBot-VA) use the prediction at test time, while GR-1, WorldVLA and pico-WAM use it only to shape the representation during training. pico-WAM keeps every decision at the smallest scale that still exposes it, two to four orders of magnitude below the large systems. Entries summarize each paper's main configuration.

Three of these systems report lessons that our small model reproduces. GR-1 finds that predicting a few frames ahead helps more than predicting the next frame [11]; pico-WAM shows the same horizon trade-off (Section 4). Cosmos Policy fine-tunes its planning model on policy rollouts, failures included, before trusting its predictions [6]; pico-WAM's world model also needs non-expert actions. V-JEPA 2-AC plans in representation space and avoids generating pixels [1]; in pico-WAM, pixels are where most of the cost goes (Section 5).

3 Building pico-WAM

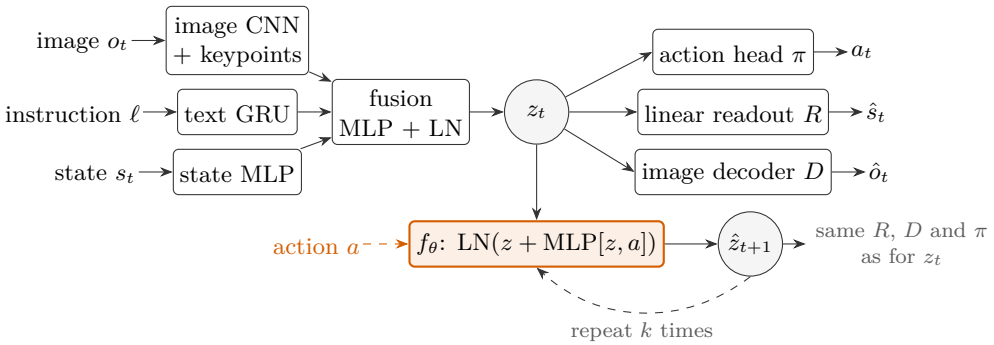


Figure 3: pico-WAM in one picture. Three small encoders and a fusion MLP turn the image, the instruction and the robot state into one latent z_t. Everything else reads from that latent: the action head (MLP, diffusion or flow), a *linear* state readout, and an image decoder. The world model (orange) is a residual MLP that maps (z, a) to the next latent; LayerNorm keeps imagined latents on the scale of encoded ones. Applying f_θ repeatedly with any action sequence "imagines" the future, and the same readouts turn every imagined latent back into a state and a picture. Parameter counts per module appear in Figure 12a.

Architecture. Figure 3 shows the full model. We keep the encoder of a small VLA and add three modules: the latent dynamics f_θ, a linear readout R and an image decoder D. A linear readout makes the latent interpretable by construction: if position and velocity are linear functions of z, the rows of R say which directions encode them. The image encoder ends in a spatial softmax, which turns each of 16 feature maps into an (x, y) keypoint we can draw.

One training step. We sample windows of H steps from one episode, encode only the first observation, and roll the dynamics forward with the actions that were actually taken (Figure 4). The loss is

$$\mathcal{L} = \mathcal{L}_{\text{act}} + w_s\,\mathcal{L}_{\text{state}} + w_i\,\mathcal{L}_{\text{image}} + w_z\,\frac{1}{H}\sum_{k=1}^{H}\left\|\hat{z}_{t+k} - \text{sg}[\text{enc}(o_{t+k})]\right\|^2, \tag{1}$$

where the state and image terms compare readouts of the encoded and imagined latents with the true current and future states and frames, and sg stops gradients through the targets. We use $w_s = 0.5$, $w_i = w_z = 1$ and $H = 5$. Removing every term except $\mathcal{L}_{\text{act}}$ gives the VLA baseline, with identical encoders.

A toy world built for checking. We train on a 2D point mass with momentum ($v_{t+1} = 0.8v_t + 0.01a_t$) and two goals; the instruction names the goal (Figure 5). The six-number state is fully known, a true simulator answers counterfactual questions, and 400 episodes collect in seconds. Velocity never appears in an image, so a single frame cannot reveal it. A PD expert provides demonstrations, and short bursts of random actions are mixed in. We tag every transition, so the policy imitates only the expert while the world model learns from everything.

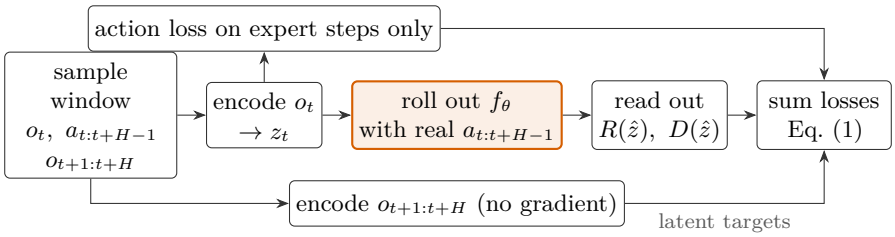


Figure 4: One training step. A window of $H = 5$ transitions is drawn from a single episode. Only the first frame is encoded with gradients. The dynamics then imagines the next H latents from the recorded actions, without seeing $o_{t+1}, \ldots, o_{t+H}$. Readouts of the imagined latents are compared with the true future states and frames, and the imagined latents are pulled toward stop-gradient encodings of the real future (latent consistency). The policy imitates only expert actions, while the world model learns from every transition, including random-action bursts. All data sits on the GPU as tensors, so a batch is built by indexing rather than by a per-sample loader.

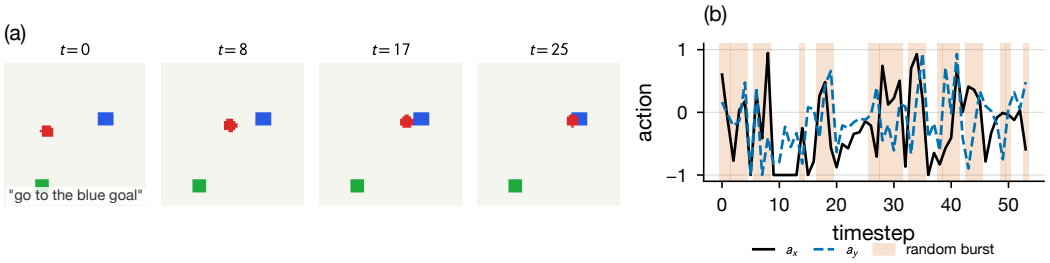


Figure 5: The toy world. (a) Frames rendered by the environment during one expert episode. A red point mass must reach the goal named in the instruction; the other square is a distractor that appears only in the image. Momentum makes velocity part of the state, but no single frame shows it. (b) Actions in one training episode. At each step the collector starts, with probability 0.3, a burst of 1–3 uniformly random actions (shaded); all other actions come from the expert. The bursts teach the world model what actions do; the policy loss ignores them.

Protocol. Each configuration runs three times, each with its own data, split and initialization. We report mean ± standard deviation across runs and show each run as a dot. Imagination is evaluated on held-out episodes: we encode one frame, roll the dynamics forward, read out the state and compare it with the truth. For *factual* error we use the recorded actions. For *counterfactual* error we use random action sequences and compare with the simulator.

4 What worked, and what did not

Table 2 summarizes every design decision we tested. The figures that follow show the most informative ones.

The latent world model imagines states well, pixels poorly. Fifteen steps into an open-loop rollout, state error is 0.0071 ± 0.0004, about $80\times$ below persistence; under random actions the 10-step error is 0.0072, versus 0.388 for ignoring the action (Figure 6). The decoded frames tell a different story: they stay worse than simply copying the

Decision	What we tried	Outcome	Verdict
Training data	expert only vs. 30% random bursts	counterfactual error at k=10 falls from 0.112 to 0.0072, even though factual error is lower with expert-only data (Fig. 8)	**worked**: diverse actions
Imagination horizon H	1, 5, 10 steps	10-step counterfactual error 0.0139, 0.0072, 0.0057; one-step error 0.0019, 0.0019, 0.0030	**trade-off**: train for the horizon you use
Latent consistency	on vs. off	15-step error 0.0071 ± 0.0004 vs. 0.0096 ± 0.0024; one-step error slightly better without it	**worked**: stability
Image loss	on vs. off (state input)	15-step state error 0.0071 vs. 0.0074; no measurable effect	**not needed** with state input
Decoder output	sigmoid vs. residual on the mean image	sigmoid stalled below the mean-image baseline; residual learns, yet imagined frames stay worse than copying the frame (Fig. 6c)	**partly**: pixels remain weak
Image encoder	global pooling vs. spatial-softmax keypoints	with state input the keypoints barely move; without it they track the agent ($R^2 \geq 0.998$) (Fig. 10)	**worked** only without state input
Diffusion head	linear schedule ($\bar{\alpha}_T \approx 0.92$) vs. cosine ($\bar{\alpha}_T \approx 10^{-5}$)	the linear schedule never reaches noise, yet sampling starts from noise; fixed with a cosine schedule and the DDPM posterior	**bug fixed**
Auxiliary supervision	BC, + current state, + future state (pixel input)	success 17.0%, 28.4%, 33.6%; most of the gain comes from current-state supervision (Fig. 11c)	**mostly state supervision**
Few demonstrations	25 to 400 episodes	WAM trails VLA at 25 episodes (87% vs. 99.7%); matched updates close most of the gap	**undertraining**

Table 2: Design decisions we tested and what happened. Errors are normalized state MSE on held-out episodes (lower is better); success is closed-loop success on 100 unseen episodes per run. Values are means over three independent runs; the supervision pilot uses five. "Factual" rollouts use the recorded actions; "counterfactual" rollouts use random actions checked against the true simulator.

current frame at every horizon. Figure 7 shows why: the decoder draws the agent and the instructed goal but loses the distractor.

Expert-only data hides action effects. Trained on expert demonstrations alone, the world model predicts recorded trajectories *better* than with exploration, yet its counterfactual error is 15× worse (Figure 8). In expert data the action follows from the state, so a network can fit the future without reading its action input, a world-model form of causal confusion [3]. The data still identifies the dynamics: an affine model of state and action, given the known clipping rule, fits the transition to numerical precision from deterministic expert trajectories alone. The failure therefore lies in what the neural model learns, not in what the data contains. A few random bursts fix it.

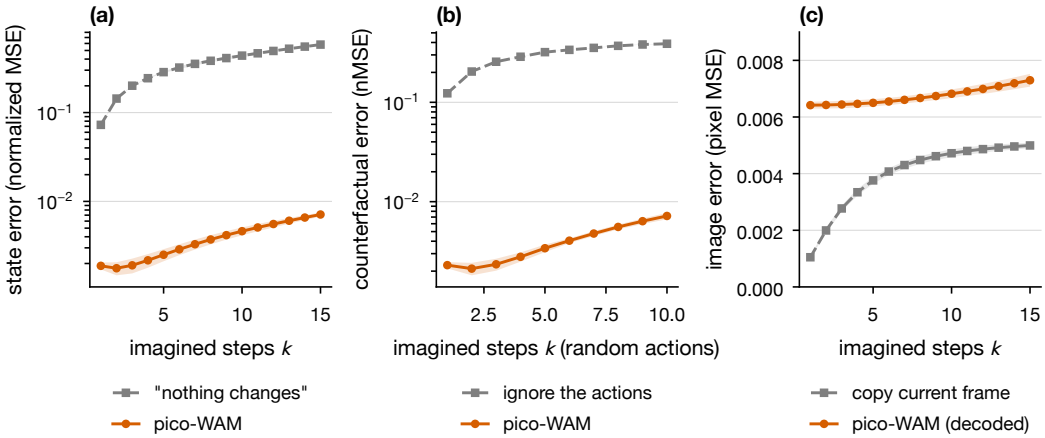


Figure 6: Imagined states are accurate and action-aware; imagined pixels are not. (a) Factual state error versus horizon for pico-WAM and a "nothing changes" baseline that repeats the current state. (b) Counterfactual error under random action sequences, compared against the simulator, versus a reference that ignores the actions. (c) Pixel MSE of decoded imagined frames versus copying the current frame, which is a strong baseline because most pixels are static background. Lines are means and bands one standard deviation over three runs; 40 held-out episodes per run.

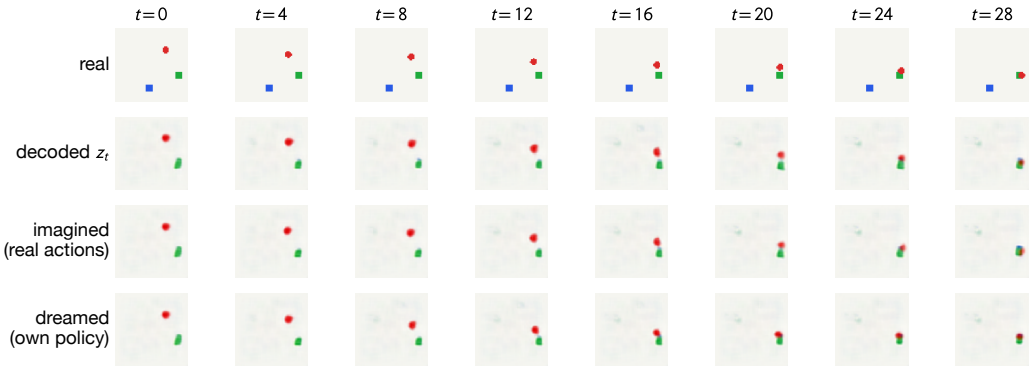


Figure 7: One held-out episode through the model's eyes. Rows show the real frames, the decoding of each real frame's latent, an open-loop imagination from $t = 0$ with the recorded actions (the model never sees frames 1–28), and a "dream" in which the model's own policy chooses the actions inside its imagination. The agent's trajectory survives 28 imagined steps; the distractor goal does not survive decoding at all.

The model extrapolates, imperfectly. Asked to push in one direction for 15 steps, far longer than any training burst, pico-WAM moves the agent the right way but stops short (Figure 9). Its endpoint error is 0.103 ± 0.010 arena units, against 0.457 for a prediction that ignores the push.

Keypoints learn to look only when they must. With the state as input, the 16 spatial-softmax keypoints sit almost still near the image center. When the state is removed, they lock onto the agent and the goals (Figure 10). An encoder learns to localize only when nothing else tells the network where things are.

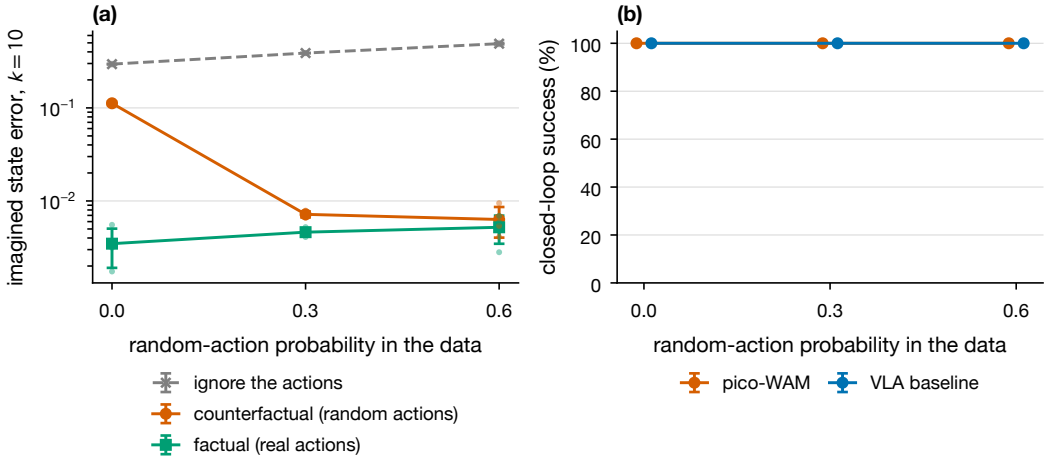


Figure 8: Factual accuracy does not imply that the model understands actions. (a) State error after 10 imagined steps as the share of random-action bursts in the data grows. With expert-only data the factual error (green, recorded actions) is lowest, while the counterfactual error (red, random actions checked against the simulator) is 15× higher than with 30% bursts. Grey: the error of ignoring the actions. (b) Closed-loop success is unaffected because the policy imitates only expert steps. Dots are individual runs; bars are one standard deviation over three runs. Exploration conditions also change trajectory distributions, so this is a comparison of datasets rather than a controlled causal estimate.

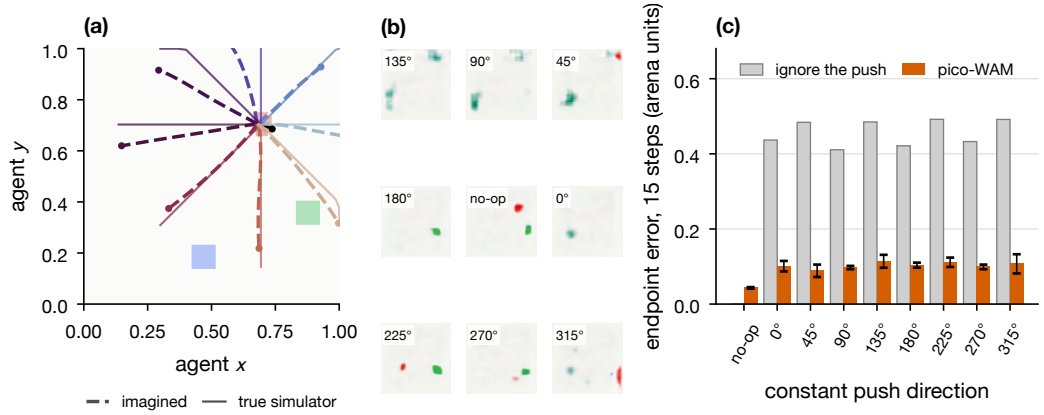


Figure 9: "What if I pushed this way?" (a) From one held-out start, imagined (dashed) and simulated (solid) trajectories under 15 steps of constant force in eight directions and no force. (b) Decoded imagined frames after 15 steps, arranged by push direction. (c) Endpoint error of the imagined agent position, averaged over the first frame of all 40 held-out episodes per run; bars are means and whiskers one standard deviation over three runs. Grey bars show the error of predicting the no-push endpoint for every push.

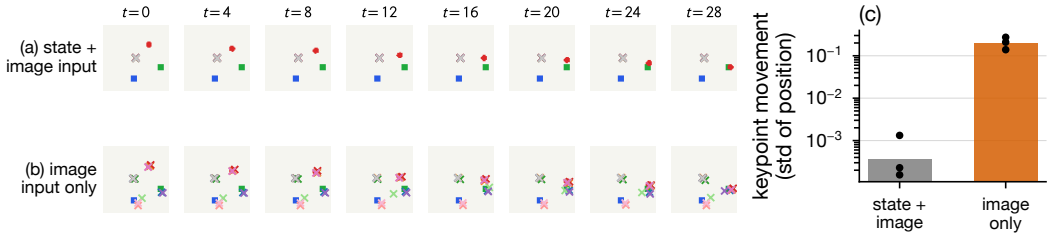


Figure 10: The image encoder localizes only when it has to. Spatial-softmax keypoints (crosses, one colour each) on a held-out episode for (a) a model that also receives the state, where the keypoints barely move, and (b) a model that receives only the image, where they follow the agent and mark both goals. (c) Spread of keypoint positions over held-out episodes on a log scale; bars are geometric means, dots individual runs. The best keypoint tracks the agent with $R^2 \geq 0.998$ in every image-only run.

When prediction helps the policy. With the state as input, the WAM never meaningfully beats the VLA, and with 25 demonstrations it trails until training gets more updates (Figure 11a). With pixels only, the WAM wins every run, $80.3 \pm 12.0\%$ versus $36.0 \pm 19.8\%$ (Figure 11b). A matched pilot separates the ingredients: supervising the current state lifts behavioral cloning from 17.0% to 28.4%, and adding future-state prediction reaches 33.6% (Figure 11c). Most of the gain is state supervision; predicting the future adds a smaller, less consistent benefit.

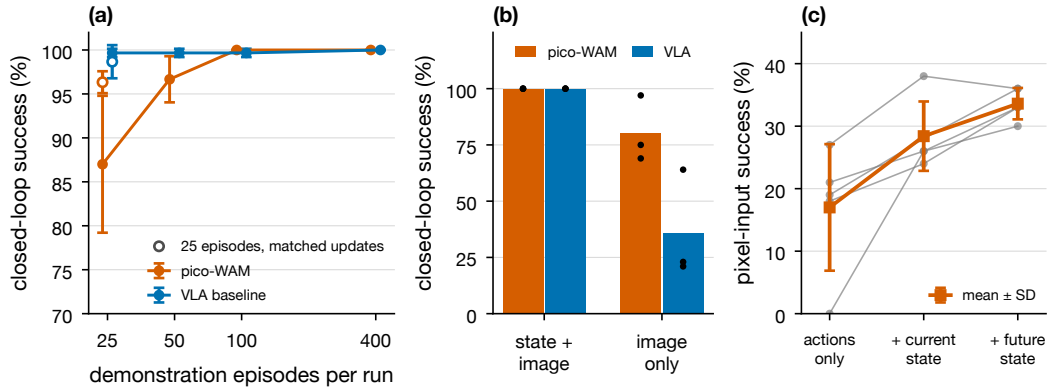


Figure 11: Prediction pays off when the observation hides the state. (a) Closed-loop success versus the number of demonstrations, 15 epochs each; open markers train the 25-episode datasets for 225 epochs to match the update count of the 400-episode runs. (b) Success with state and image input versus image input only (400 episodes); dots are individual runs. (c) A matched pixel-input pilot with identical data, initialization, batches and update counts: behavioral cloning alone, plus current-state supervision, plus five-step future-state prediction. Grey lines connect the conditions of each of five runs; orange shows mean ± sample SD. Panels (a, b) use three runs and the full WAM objective; (c) excludes pixel and latent-consistency losses.

5 Where the cost goes

Figure 12 profiles the trained model. Acting costs the same for WAM and VLA, 0.42 ms per step, because the deployed policy never calls the dynamics. Training is a different

story. One WAM step costs 38× a VLA step, and the image decoder accounts for most of it: removing it cuts the step from 2.55 s to 0.27 s and halves the parameters. Imagination itself is cheap. Ten latent steps with state readouts take 0.28 ms; decoding a frame at every step raises that to 12.7 ms. Three changes made pico-WAM fast enough to run 12 three-run experiments in an afternoon: predict states rather than pixels where possible, keep the dataset on the GPU and index batches instead of looping over samples (4.5 ms to 2.6 ms per batch even on a CPU), and roll out in latent space.

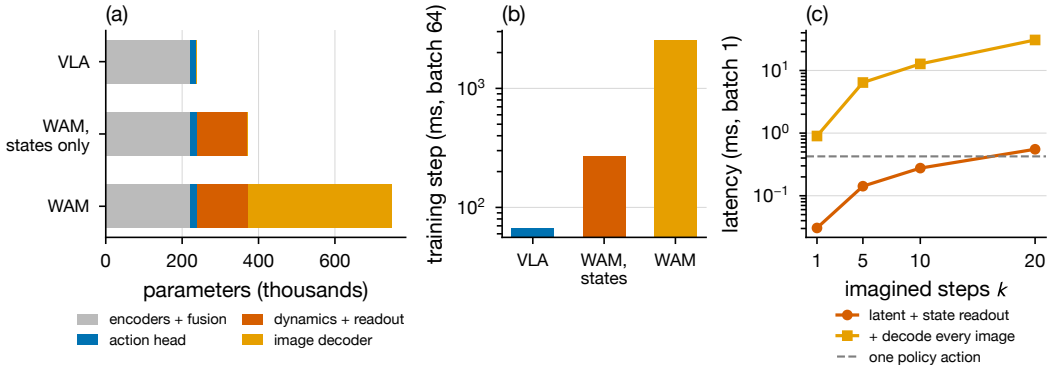


Figure 12: Pixels dominate the cost of a small WAM. (a) Parameters per module for the VLA baseline, a WAM that predicts only states, and the full WAM; the image decoder holds half of the full model. (b) Wall time of one optimizer step at batch size 64 (log scale). Dropping the decoder makes a WAM step 9.5× cheaper. (c) Latency of imagining k steps from one observation, reading out states only or also decoding every frame, compared with the cost of one policy action (dashed). Medians of repeated calls on one CPU thread of an Apple-silicon laptop; the numbers compare components with each other, not hardware.

6 Limitations

pico-WAM is a teaching and research instrument, not a robot controller. The toy world is fully observed when the state is given, its dynamics are affine away from the walls, and its goal coordinates appear in the state. The policy never uses imagination at test time, so none of our results measure planning. In the pixel-input setting the WAM receives privileged state targets during training, and a history-based baseline is still missing; velocity is hidden in a single frame, and a short frame history may close much of the gap. Our studies use three runs per configuration and five in the pilot, which is enough to see large effects but not small ones.

7 Conclusion

A WAM is a VLA with a learned simulator that shares its representation. Built from scratch at 0.75M parameters, that simulator imagines accurately, reads out linearly and responds to actions, provided the data contains actions worth responding to. Our experiments point to three practical rules. Collect diverse actions for the world model, even while the policy imitates the expert. Expect prediction to help the policy when the observation hides the state, and test whether plain state supervision explains the gain.

Predict compact states or latents before pixels, which is where most of the cost lies. The large open WAMs in Table 1 make the same choices at a scale where they are expensive to test; pico-WAM makes them cheap to test.

Reproducibility. Every figure is regenerated from committed result files by `docs/figures/make_figures.py`; every experiment is `scripts/run_toy.sh` with one configuration line changed. Code: `https://github.com/keivalya/pico-wam`.

Assistance disclosure. AI assistance was used to audit code, develop diagnostic experiments, check literature and venue information, and revise this manuscript. The results presented as fresh measurements were executed; archived results are labeled as such. Author review of the scientific claims and submission-specific disclosure remains necessary.

References

[1] M. Assran, A. Bardes, D. Fan, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv:2506.09985, 2025.

[2] J. Cen, C. Yu, H. Yuan, et al. WorldVLA: Towards autoregressive action world model. arXiv:2506.21539, 2025.

[3] P. de Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. NeurIPS, 2019. arXiv:1905.11979.

[4] S. Huang, P. Kaushik, M. Chen, et al. Nano World Models: A minimalist implementation of future video prediction. arXiv:2605.23993, 2026.

[5] J. Jang, S. Ye, Z. Lin, et al. DreamGen: Unlocking generalization in robot learning through video world models. arXiv:2505.12705, 2025.

[6] M. J. Kim, Y. Gao, T.-Y. Lin, et al. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. arXiv:2601.16163, 2026.

[7] L. Li, Q. Zhang, Y. Luo, et al. Causal world modeling for robot control (LingBot-VA). arXiv:2601.21998, 2026.

[8] Y. Liao, P. Zhou, S. Huang, et al. Genie Envisioner: A unified world foundation platform for robotic manipulation. arXiv:2508.05635, 2025.

[9] Z. Lu, H. Zhai, et al. World-action models for robot learning and control: A survey. arXiv:2609.16074, 2026.

[10] Q. Shen, S. Zhang, Y. Liao, et al. World action models: A survey. arXiv:2606.20781, 2026.

[11] H. Wu, Y. Jing, C. Cheang, et al. Unleashing large-scale video generative pre-training for visual robot manipulation (GR-1). arXiv:2312.13139, 2023.

[12] X. Zhang, X. Zeng, and W. Zhang. From world models to world action models: A concise tutorial for robotics. arXiv:2607.00836, 2026.

[13] C. Zhu, R. Yu, S. Feng, et al. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. RSS, 2025. doi:10.15607/RSS.2025.XXI.015.